UNIVERSITY OF St.Thomas

SEIS 606: Vibe Coding

Fall 2026

Paul Kaefer, Adjunct Professor

Tonight's Agenda

1. introductions
2. syllabus review
3. software engineering landscape: before AI & now
4. git/GitHub crash course

Introducing Your Professor

Adjunct Professor at the University of St. Thomas

Courses: Data Analytics & Visualization and Vibe Coding

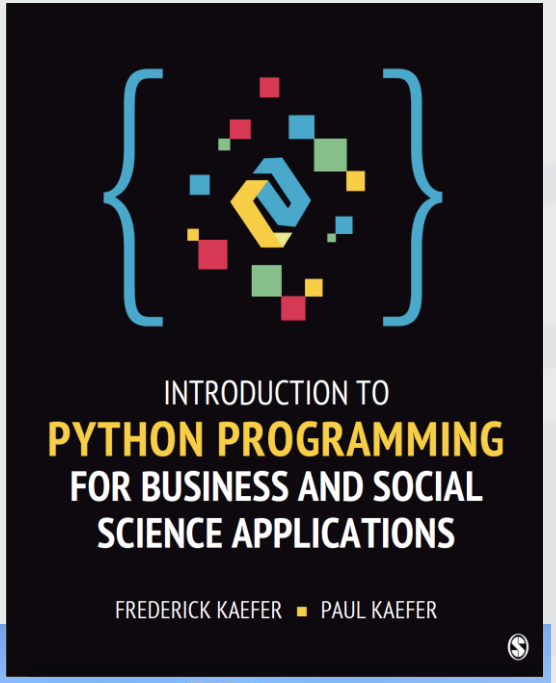
Senior Data and AI Enablement Analyst at RBC Wealth Management

~10 years in healthcare analytics

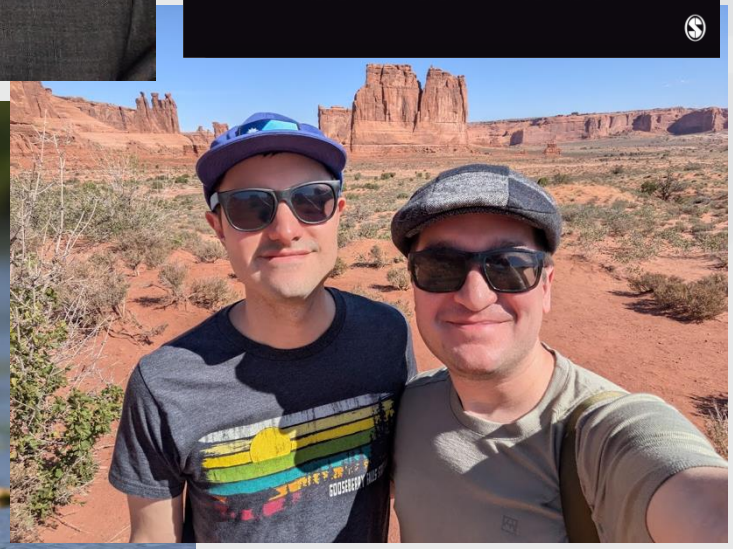
M.S. in Computational Sciences from Marquette University

Hobbies include reading, traveling (18 countries so far), and wildlife photography









Please introduce yourself!

Preferred name & anything you would like us to know

Already have an app idea? Or a hobby you'd like to build something for?

What's the coolest thing you've built? Could be tech, crafts, etc.

A disclaimer before we continue...

I will use generative AI tools (notably Claude)
for ideas and wording for my lectures.

I wrote the syllabus and created this slide deck myself.

UNIVERSITY OF St.Thomas

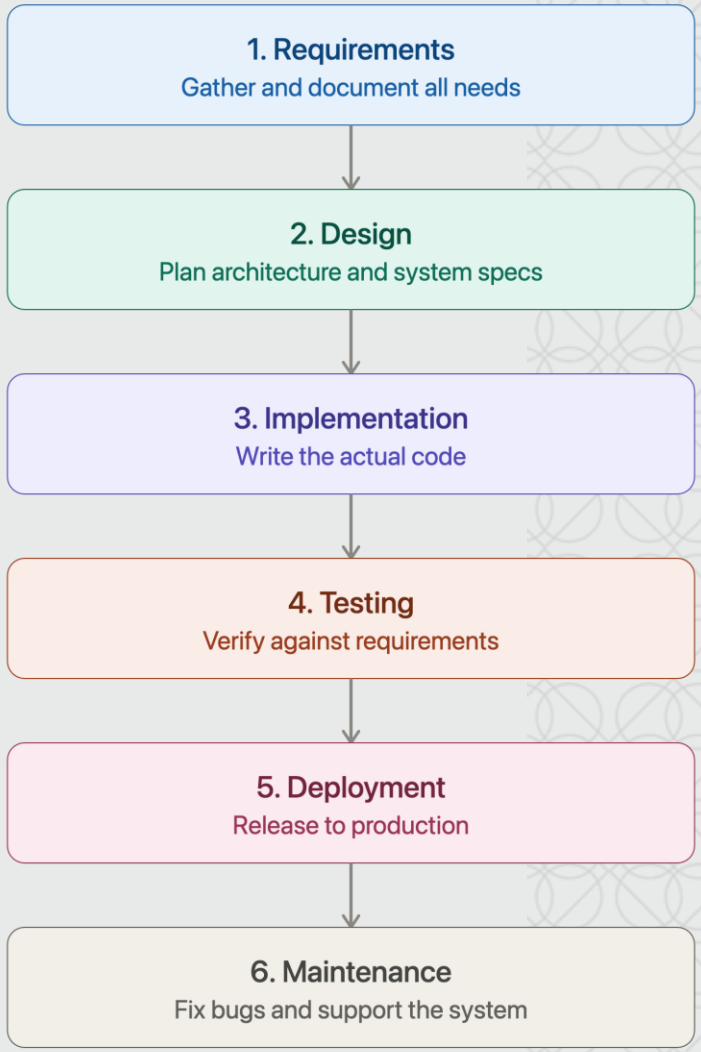
First, a quick poll...

Software Engineering

Traditionally, in phases starting with analysis/requirements gathering.

Design involves architecture for system as a whole, and broken down into individual features.

Code is written to follow the specifications.



Waterfall method
(image generated using Claude)

References: Wikipedia and *Encyclopedia of Computer Science and Technology* (Harry Harrison, 2009)

More modern agile approach

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

See "Manifesto for Agile Software Development"

What does agile look like?

Work divided into chunks and prioritized into sprints (usually about two weeks).

Various **ceremonies** including [daily] standups, sprint reviews/demo, and backlog refinement.

Very iterative!

Very iterative

CI/CD: continuous integration/continuous deployment (or "delivery")

- While not applicable (or appropriate) for every software product, this is done in Big Tech, for apps you use like Netflix, Spotify, Google, etc.
- Tools like GitHub Actions and Amazon Web Services (AWS) allow software to be automatically tested and deployed (if tests pass!)

Test-Driven Development (**TDD**) is one approach that goes hand-in-hand.

- Write tests first! Then code that makes the tests pass.
- Counterintuitive? Or brilliant!

The book *Test Driven Development: By Example* (Kent Beck, 2002) is an excellent resource!

Whatever you do, start with the design.

Who are your stakeholders?

What is your scope?

Write **user stories** based on answers to these questions, to drive development.

The equivalent to tests: **acceptance criteria**. How do you define "done"?

A short break before we "introduce" AI...

UNIVERSITY OF St.Thomas

You are smarter than AI.

AI is not sentient

Large Language Models (LLMs) use a lot of text, but are ultimately *predicting* or *generating*, based on probabilities.

Think of these tools like…

1. A *really* fast intern, prone to mistakes (including hallucinations!)
2. A search engine that can also write
3. A blueprint generator, not the finished building

AI tools need human oversight!

Here's another way to describe generative AI:

> A powerful tool, especially in the hands of a critical thinker.

We'll talk more about human-in-the-loop in a bit!

Writing a Good Prompt

Similar to Googling, there are tips to get the most out of prompting.

In essence, both web search and prompting are about crafting good questions.

Two frameworks I'll cover:

- **CRAFT** (coming up with a single prompt)
- **TCREI** (Google's prompting framework for the whole process)

CRAFT framework for prompt writing

Context: provide information on your task, including background, field/domain, and constraints.

Role: tell the tool to take on a role, which can include the desired audience.

Action: let it know the task(s), including specific steps, and quality standards.

Format: what do you want the results to look like?

Target Audience: who will be impacted by this, and what is their knowledge/skill level?

CRAFT example #1: Paul's use case

Context: I am developing a "vibe coding" course.

Role: Take on the role of an experienced professor who wants their students to succeed, both inside and outside of the classroom.

Action: Generate me an outline for a course, as well as a list of concepts I should be sure to cover.

Format: The outline should be a list that uses Roman numerals, and the questions should be a bulleted list.

Target Audience: While this is my first time working with most of these students, I assume there is a wide range of coding skills, and comfort levels/experience using AI.

CRAFT example #2: presenting to leadership

Context: I am presenting quarterly business statistics to my leadership.

Role: You are an expert in communication who will not hesitate to point out any gaps or inconsistencies in my work.

Action: Using the attached spreadsheet, create some summary statistics for my presentation.

Format: The stats should be in a table I can use in my report.

OR: produce an HTML file with a few simple visuals from the data.

Target Audience: My leaders care about accuracy and will ask detailed questions about any unusual or surprising figures.

Shall we try a few prompts?



Always Be Iterating...

Google's TCREI framework takes this further...

Task or persona: this includes role & format like we discussed previously

Context: what you need from the tool (maybe "15 ideas" or "a budget plan")

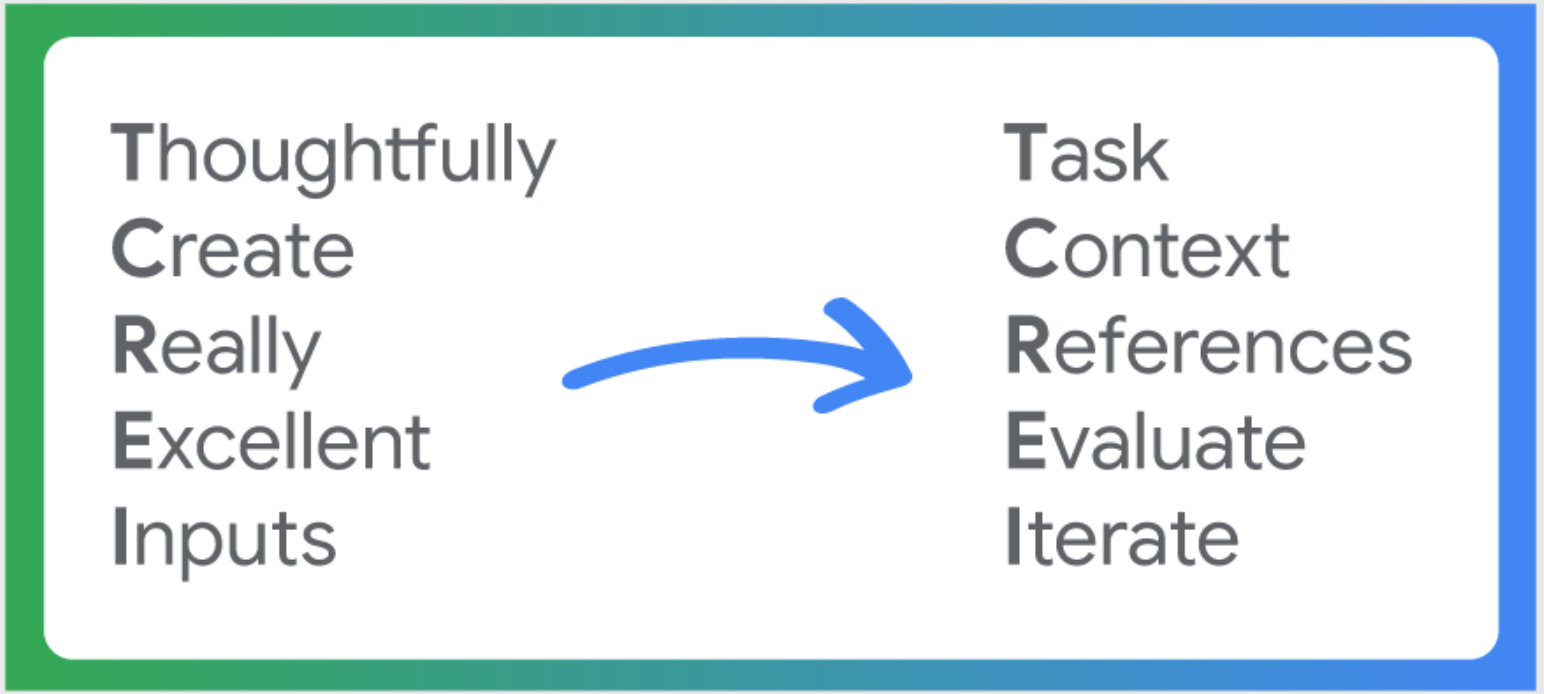
References: aim for 2-5 past examples that the tool can "learn" from

Evaluate: did the input you provided produce an output meeting your need?

Iterate: add information/tweak your prompt until you can answer "yes!"

Source: Google's Start Writing Prompts Like a Pro course (Coursera)

There's a mnemonic for this!



Source: Google's Start Writing Prompts Like a Pro course (Coursera)

Additional Resources

Prompt Engineering overview from Anthropic

Two prompt engineering guides (fairly technical; recommended for those who like to go "down the rabbit hole"):

- brexhq/prompt-engineering
- dair-ai/Prompt-Engineering-Guide

"A High-level Overview of Large Language Models." RBC Borealis. July 2023.

- Very technical deep-dive into LLMs and research.



Now we'll get a little more technical...

Prompting Strategies

Zero-shot: asking the tool to do something with no examples

Write an email to my professor with a good excuse about why I can't do my homework this week.

One-shot: provide one example

Here's an example of a vibe-coded app idea. Now write one for...

Few-shot: provide multiple examples

Role playing/persona: essentially what we described in the CRAFT framework

Act as an experienced software developer. Poke holes you see in my attached architecture document/spec.

More prompting strategies

Chain-of-thought: ask the AI tool to show it's reasoning, step-by-step

Explain your reasoning as you go.

Prompt chaining: breaking a complex task into sequential prompts.

Note: this works in the same thread.

Prompt 1: Summarize the report/email as currently written.

Prompt 2: Identify risks…

Prompt 3: Recommend fixes to the risks you identified.

…

Whatever you do…

Highly recommend saving your prompts outside of whatever tool(s) you use.

This can be as simple as a file on your computer, or something like a Google Doc.

Benefits:

- Build-up a catalog or library for future use
- Don't rely on these tools saving your history indefinitely
- Also can centralize prompts used across AI tools
- Makes it easy to copy/paste, refine, and share!

Considerations as we start bringing in AI…

Human-in-the-loop

Hallucinations happen: false or misleading information that AI presents as fact.

The main way to counteract this is to always ensure a human is reviewing outputs (including code).

Remember the comparison of AI tools to interns? Would you ever push work an intern did to production without a senior developer reviewing it?

Concerns about AI: my thoughts

Always provide a disclaimer when you use output from generative AI.

Don't use generative AI as your own work.

Be cautious about using generated imagery.

- There's great value in paying a human artist for logos, marketing materials, etc.
- There are a lot of AI-generated posters out there that just look tacky!

Environmental concerns

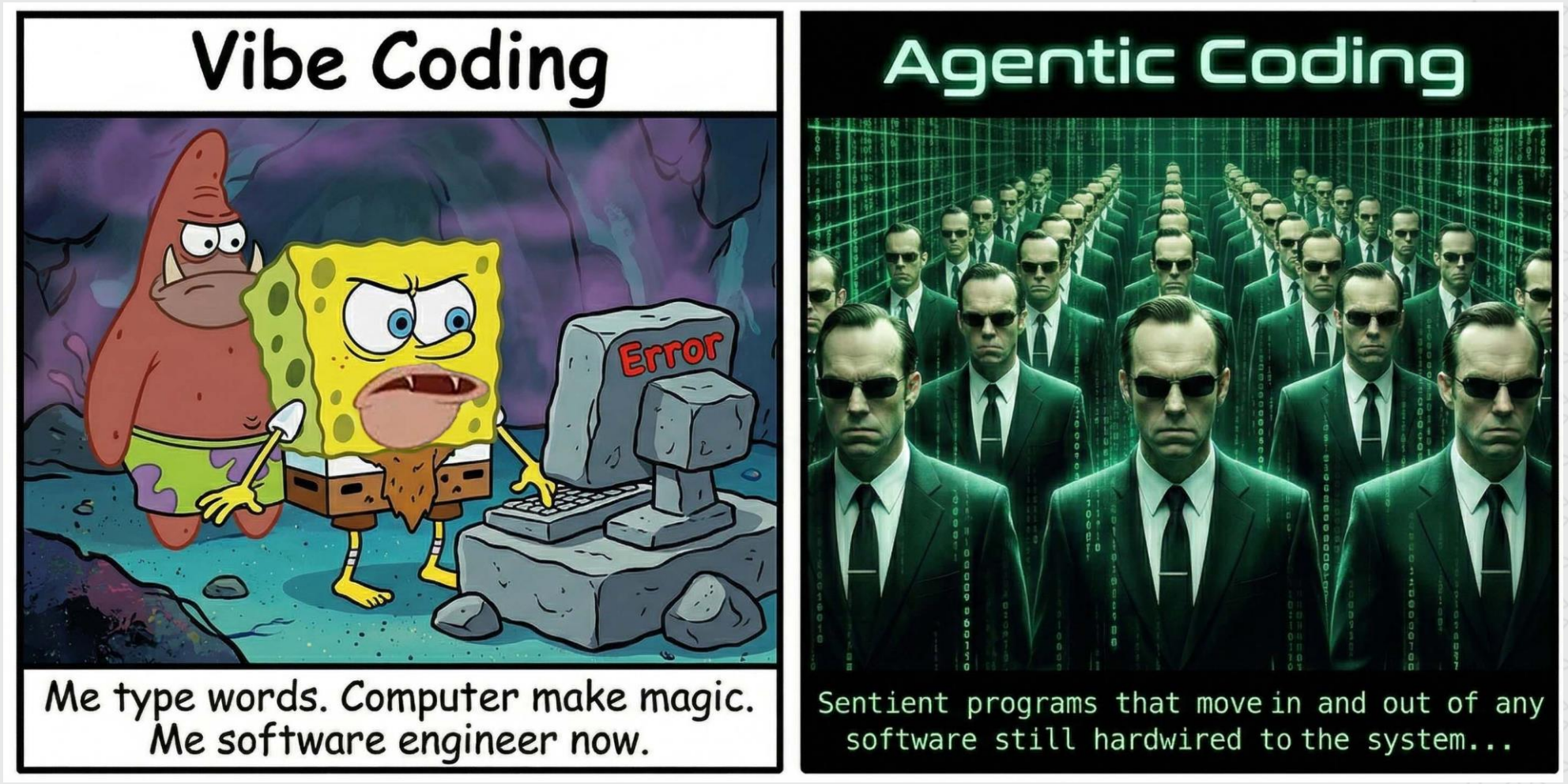
For some people, this is enough of a reason to avoid using AI tools whenever possible.

Consider real actions you can take, such as using alternative tools and voicing your opinions. This could include asking for an environmental impact study for any tools you are evaluating.



Now are we ready to talk "vibe coding"??

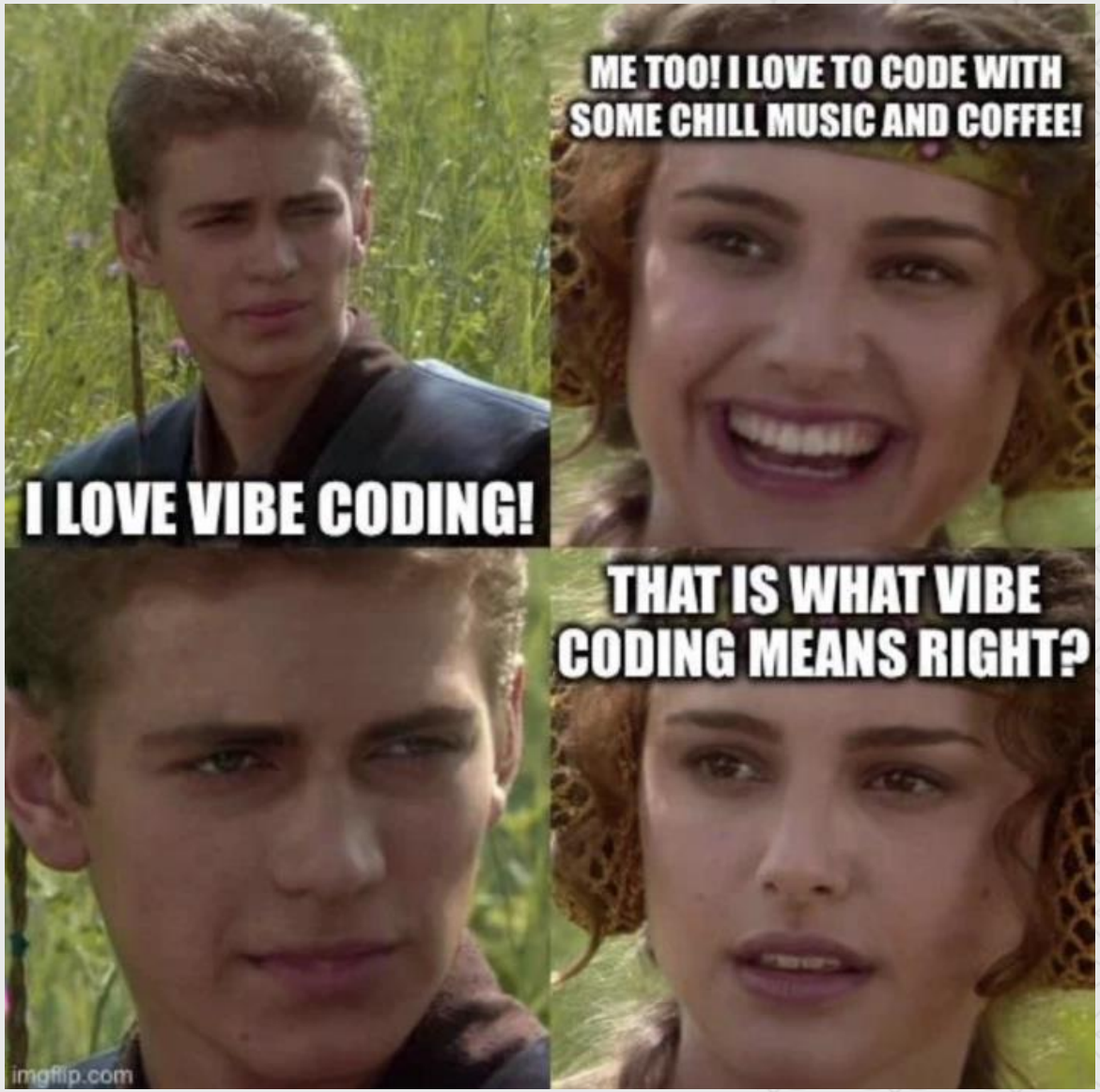
Regarding the title of the class...



Found on Facebook

More memes!





credit



Lab: git/GitHub

Getting started with GitHub

1. Sign-up at GitHub.com (I recommend using your @stthomas.edu email)
   a) If you already have an account, add your @stthomas.edu at https://github.com/settings/emails

2. On Windows, download https://git-scm.com/install/windows

3. Optional: GitHub Desktop, https://desktop.github.com/download/

4. Also become verified for free GitHub Copilot access as a student https://docs.github.com/en/copilot/how-tos/copilot-on-github/set-up-copilot/enable-copilot/set-up-for-students

Git resources

"Cheat sheet" @ https://git-scm.com/cheat-sheet

Another one @ https://github.com/0nn0/git-basics-cheatsheet

git game: https://github.com/git-game/git-game



Homework #1

1. Setup an account on GitHub
2. Push several "vibe code" app ideas to a repo
3. Also apply to GitHub Copilot for students

I recommend applying for GitHub Copilot for students as soon as you can!

UNIVERSITY OF St.Thomas

Any questions before we end?